R documentation

of all in 'man'

July 18, 2026

Contents

`bnb_elasticities` *Elasticities and semi-elasticities for a bivariate NB model*

Description

For continuous regressors, the elasticity is $\beta_j E[x_j]$; for binary (0/1) regressors, the semi-elasticity is $\exp(\beta_j) - 1$ (proportional change moving from 0 to 1).

Usage

```
bnb_elasticities(
  fit,
  which = c("y1", "y2", "both"),
  type = c("AME", "MEM"),
  vars = NULL,
  include_intercept = FALSE,
  digits = 4,
  print_output = TRUE
)
```

Arguments

fit	A bnb_fit object from fit_bnb().
which	Which margin(s): "y1", "y2", or "both".
type	"AME" (uses sample mean of x) or "MEM" (uses mean design row).
vars	Optional variable names to restrict output.
include_intercept	Logical; include the intercept term.
digits	Number of decimal places for printed output.
print_output	Logical; if FALSE, suppress printing.

Value

A data frame (single margin) or a named list of data frames (both), each with columns Name, Estimate, StdErr, z, p, Signif, var_type.

Examples

```
d <- read.csv(system.file("extdata", "rwm1984_clean.csv", package = "rpbnb"))
fit <- fit_bnb(docvis ~ outwork + age, hospvis ~ outwork, data = d,
               dependence = "famoye")
bnb_elasticities(fit, which = "both", type = "AME")
```

---

bnb_gof *Goodness of fit for a bivariate NB model*

Description

Reports log-likelihood, AIC, BIC, and four pseudo-R-squared measures (McFadden, McFadden adjusted, Cox-Snell, Nagelkerke) relative to an intercept-only null model refit with the same dependence structure (copula fits rebuild the copula from fit$cop_family). Pseudo-R-squared values are returned raw and are not clamped to [0, 1]; a negative value flags a full model that fits worse than the null.

Usage

```
bnb_gof(fit, digits = 4, print_output = TRUE)
```

Arguments

fit	A bnb_fit object from fit_bnb().
digits	Number of decimal places for printed output.
print_output	Logical; if FALSE, suppress printing and return the result invisibly.

Value

Invisibly, a list with n, k, logLik_full, logLik_null, AIC, BIC, a named numeric vector pseudoR2, and the null_fit.

Examples

```
d <- read.csv(system.file("extdata", "rwm1984_clean.csv", package = "rpbnb"))
fit <- fit_bnb(docvis ~ outwork + age, hospvis ~ outwork, data = d,
               dependence = "famoye")
bnb_gof(fit)
```

---

bnb_marginal_effects *Marginal effects for a bivariate NB model*

Description

Average marginal effects (AME) or marginal effects at the mean (MEM) for each margin. Continuous and binary (0/1) regressors are auto-detected; continuous effects use $\partial E[Y]/\partial x_j = \beta_j \mu$ and binary effects use $E[Y|x_j = 1] - E[Y|x_j = 0]$.

Usage

```
bnb_marginal_effects(
  fit,
  which = c("y1", "y2", "both", "all"),
  type = c("AME", "MEM"),
  vars = NULL,
  include_intercept = FALSE,
  digits = 4,
  print_output = TRUE
)
```

Arguments

fit	A bnb_fit object from fit_bnb().
which	Which margin(s): "y1", "y2", "both", or "all".
type	"AME" (average marginal effect) or "MEM" (effect at the mean).
vars	Optional variable names or indices to restrict output.
include_intercept	Logical; include the intercept term.
digits	Number of decimal places for printed output.
print_output	Logical; if FALSE, suppress printing.

Value

A data frame (single margin) or a named list of data frames (both), each with columns `Name`, `Estimate`, `StdErr`, `z`, `p`, `Signif`, `var_type`.

Examples

```
d <- read.csv(system.file("extdata", "rwm1984_clean.csv", package = "rpbnb"))
fit <- fit_bnb(docvis ~ outwork + age, hospvis ~ outwork, data = d,
               dependence = "famoye")
bnb_marginal_effects(fit, which = "y1", type = "AME")
```

`bnb_residual_checks` *Residual checks for a bivariate NB model*

Description

Formal residual diagnostics for a `fit_bnb()` or `fit_rpbnb()` fit: a normality test on the randomized quantile residuals (Shapiro-Wilk for `n <= 5000`, else Kolmogorov-Smirnov vs N(0,1)); the NB2 dispersion statistic (`sum(pearson^2) / (n - k)`) per margin; the cross-margin RQR correlation (a check that the Famoye/copula dependence captured the association); an outlier count/index list (`|RQR| > outlier_z`); and a composite misspecification verdict combining these signals.

Usage

```
bnb_residual_checks(
  fit,
  seed = NULL,
  outlier_z = 3,
  digits = 4,
  print_output = TRUE
)
```

Arguments

`fit`	A `bnb_fit` or `rpbnb_fit` object.
`seed`	Optional integer seed for the RQR randomization.
`outlier_z`	Absolute-RQR threshold flagging an outlier (default 3).
`digits`	Decimal places for printed output.
`print_output`	Logical; if `FALSE`, suppress printing.

Value

Invisibly, an object of class `bnb_residual_checks`.

copula *Specify a copula dependence structure*

Description

Pass the result as the dependence argument to `fit_bnb()` or `fit_rpbnb()` to estimate a discrete-copula bivariate NB model instead of the default Famoye/Sarmanov dependence, or as the `copula` argument to `simulate_rpbnb_copula()` to simulate from one. The joint pmf is built from the two NB2 marginal CDFs and the chosen copula CDF (rectangle differencing); see the package vignette for the "Copula dependence" section.

Usage

```
copula(family = c("frank", "normal", "kimeldorf"), par = NULL)
```

Arguments

`family`	One of `"frank"`, `"normal"` (Gaussian), or `"kimeldorf"` (Clayton).
`par`	Optional native dependence parameter (Frank's theta, the Gaussian rho, or Clayton's theta) used by `simulate_rpbnb_copula()` to generate data. Ignored by `fit_bnb()` and `fit_rpbnb()`, which estimate the parameter from the data.

Value

An object of class `rpbnb_copula`.

Examples

```
copula("frank")
copula("normal", par = 0.3)
copula("kimeldorf")
```

fit_bnb *Fit a bivariate negative binomial regression model*

Description

Fit a bivariate negative binomial regression model

Usage

```
fit_bnb(
  formula_1,
  formula_2,
  data,
  dependence = c("independence", "famoye"),
  start = NULL,
  control = rpbnb_control()
)
```

Arguments

formula_1, formula_2	Model formulas for the two count outcomes.
data	A data frame.
dependence	Dependence structure: "independence" (two univariate NB2 margins), "famoye" (Famoye/Sarmanov bivariate NB), or a copula() object (Frank / Gaussian / Clayton discrete-copula bivariate NB; the dependence parameter is estimated).
start	Optional starting parameter vector. May be positional (length equal to the number of parameters) or named; a named vector is reordered to the canonical parameter order and a named partial vector is merged into the defaults (unknown or duplicate names are rejected). When start is NULL, the famoye path uses a multi-start policy: it optimizes from both an all-zero mean-coefficient start and marginal glm.nb starts and keeps the better converged objective (the frozen-bounds gradient makes the objective start-sensitive and neither start dominates).
control	An rpbnb_control() object. The famoye and copula estimators both use BFGS, the only optimizer control$method accepts.

Value

An object of class bnb_fit.

Examples

```
d <- read.csv(system.file("extdata", "rwm1984_clean.csv", package = "rpbnb"))
fit <- fit_bnb(docvis ~ outwork, hospvis ~ outwork, data = d,
               dependence = "famoye")
summary(fit)

# Gaussian copula dependence instead of Famoye/Sarmanov
fit_cop <- fit_bnb(docvis ~ outwork, hospvis ~ outwork, data = d,
                   dependence = copula("normal"))
fit_cop$cop_tau  # estimated Kendall's tau
```

---

fit_rpbnb *Fit a bivariate random-parameter negative binomial model*

Description

Maximum simulated likelihood estimation with normal random coefficients and Famoye/Sarmanov dependence. Random coefficients are selected per equation by name via random_1 / random_2.

Usage

```
fit_rpbnb(
  formula_1,
  formula_2,
  data,
  random_1 = NULL,
  random_2 = NULL,
  draws = 400,
```

```
  draw_type = "halton",
  seed = 1234,
  start = NULL,
  control = rpbnb_control(),
  dependence = "famoye"
)
```

Arguments

formula_1, formula_2	Model formulas for the two count outcomes.
data	A data frame.
random_1, random_2	Random coefficients per equation. Either a character vector of model.matrix column names (all Normal), or a named list whose values are a distribution name ("normal", "lognormal", "uniform", "triangular") or a list list(dist = ..., sign = ...) (sign is -1/1 and lognormal-only). NULL means all-fixed for that equation.
draws	Number of simulation draws for the optimization.
draw_type	Quasi-random draw type. Only "halton" is supported in this version.
seed	Random seed for the simulation draws.
start	Optional starting parameter vector.
control	An rpbnb_control() object. Estimation uses BFGS, the only optimizer control$method accepts.
dependence	Dependence structure: "famoye" (default; Famoye/Sarmanov) or an copula() object for copula dependence (Frank / Gaussian / Clayton). Both paths use the multithreaded (OpenMP) C++ simulated likelihood; the copula path is more numerically expensive per evaluation (discrete-copula pmf + per-draw NB CDF corners), so fits typically take noticeably longer than the Famoye path at comparable draws/n. Random coefficients on 0/1 dummy regressors are weakly identified under the copula path (NB dispersion trades off against the random-coefficient scale); prefer random coefficients on continuous regressors.

Value

An object of class rpbnb_fit.

Examples

```
sim <- simulate_rpbnb(n = 600,
  beta1 = c("(Intercept)" = 0.2, x1 = 0.4),
  beta2 = c("(Intercept)" = 0.1, x1 = -0.3),
  random_1 = list(x1 = list(sd = 0.5)),
  dispersion = c(m1 = 0.4, m2 = 0.5), seed = 1)
fit <- fit_rpbnb(y1 ~ x1, y2 ~ x1, data = sim$data, random_1 = "x1",
                 draws = 100, control = rpbnb_control(compute_se = FALSE))
coef(fit)


# Copula dependence instead of Famoye/Sarmanov (slower; fewer draws here
# for a quick example -- use more in practice)
sim_cop <- simulate_rpbnb_copula(n = 600,
```

```
  beta1 = c("(Intercept)" = 0.2, x1 = 0.4),
  beta2 = c("(Intercept)" = 0.1, x1 = -0.3),
  random_1 = list(x1 = list(sd = 0.3)),
  dispersion = c(m1 = 0.4, m2 = 0.5),
  copula = copula("normal", par = 0.5), seed = 1)
fit_cop <- fit_rpbnb(y1 ~ x1, y2 ~ x1, data = sim_cop$data, random_1 = "x1",
                     dependence = copula("normal"), draws = 100,
                     control = rpbnb_control(compute_se = FALSE))
tanh(coef(fit_cop)[["z_theta"]])  # estimated copula rho
```

`plot.bnb_fit` *Residual diagnostic plots for a bivariate NB model*

Description

Four base-graphics panels per margin: residuals-vs-fitted, a normal QQ plot of the randomized quantile residuals, a histogram of the RQR with an N(0,1) overlay, and a scale-location plot. The QQ and histogram panels always use RQR (only these are approximately N(0,1) under a correct count model).

Usage

```
## S3 method for class 'bnb_fit'
plot(
  x,
  margin = c("both", "y1", "y2"),
  which = 1:4,
  resid_type = "quantile",
  seed = NULL,
  ...
)
```

Arguments

`x`	A `bnb_fit` object from `fit_bnb()`.
`margin`	Which margin to plot: `"both"` (default), `"y1"`, or `"y2"`.
`which`	Integer subset of panels `1:4` (1 = residuals-vs-fitted, 2 = QQ, 3 = histogram, 4 = scale-location).
`resid_type`	Residual type for panels 1 and 4: `"quantile"` (default), `"pearson"`, `"deviance"`, or `"response"`.
`seed`	Optional integer seed for the RQR randomization.
`...`	Unused.

Value

`NULL`, invisibly (called for the side effect of drawing).

plot.rpbnb_fit *Residual diagnostic plots for a random-parameter bivariate NB model*

Description

Four base-graphics panels per margin, as for `plot.bnb_fit()`, built on the mixture-based randomized quantile residuals. `resid_type = "deviance"` is not available for `rpbnb_fit`.

Usage

```
## S3 method for class 'rpbnb_fit'
plot(
  x,
  margin = c("both", "y1", "y2"),
  which = 1:4,
  resid_type = "quantile",
  seed = NULL,
  ...
)
```

Arguments

`x`	An `rpbnb_fit` object from `fit_rpbnb()`.
`margin`	Which margin to plot: `"both"` (default), `"y1"`, or `"y2"`.
`which`	Integer subset of panels `1:4`.
`resid_type`	Residual type for panels 1 and 4: `"quantile"` (default), `"pearson"`, or `"response"`.
`seed`	Optional integer seed for the RQR randomization.
`...`	Unused.

Value

`NULL`, invisibly.

residuals.bnb_fit *Residuals for a bivariate NB model*

Description

Per-margin residuals for a fixed-coefficient `fit_bnb()` model. Each margin is NB2 with fitted mean `mu` and dispersion `m` (`size = 1/m`). `"quantile"` returns randomized quantile residuals (Dunn & Smyth 1996), which are approximately N(0,1) under a correct model and are the recommended residual for normality-style diagnostics on count data.

Usage

```
## S3 method for class 'bnb_fit'
residuals(
  object,
  type = c("quantile", "pearson", "deviance", "response"),
  margin = c("both", "y1", "y2"),
  seed = NULL,
  ...
)
```

Arguments

object	A bnb_fit object from fit_bnb().
type	Residual type: "quantile" (default), "pearson", "deviance", or "response".
margin	Which margin: "both" (default), "y1", or "y2".
seed	Optional integer seed for the quantile-residual randomization (ignored for other types); does not disturb the caller's RNG stream.
...	Unused.

Value

A numeric vector for a single margin, or a two-column data frame (y1, y2) for margin = "both".

---

residuals.rpbnb_fit *Residuals for a random-parameter bivariate NB model*

Description

Per-margin residuals for an fit_rpbnb() model. Each margin is a mixture of NB2 distributions over the random-coefficient draws. "quantile" returns randomized quantile residuals (Dunn & Smyth 1996) from the exact mixture predictive CDF (the recommended residual); "pearson" uses the exact mixture marginal variance. "deviance" is not defined for the mixture and errors.

Usage

```
## S3 method for class 'rpbnb_fit'
residuals(
  object,
  type = c("quantile", "pearson", "deviance", "response"),
  margin = c("both", "y1", "y2"),
  seed = NULL,
  ...
)
```

Arguments

object	An rpbnb_fit object from fit_rpbnb().
type	Residual type: "quantile" (default), "pearson", or "response". "deviance" is not supported for rpbnb_fit.
margin	Which margin: "both" (default), "y1", or "y2".
seed	Optional integer seed for the quantile-residual randomization; does not disturb the caller's RNG stream.
...	Unused.

Value

A numeric vector for a single margin, or a two-column data frame (y1, y2) for margin = "both".

---

rpbnb_control *Control parameters for rpbnb estimators*

Description

Control parameters for rpbnb estimators

Usage

```
rpbnb_control(
  method = c("BFGS"),
  iterlim = 300L,
  reltol = 1e-08,
  print_level = 2L,
  draws_hessian = 100L,
  halton_burn = 300L,
  n_cores = 1L,
  compute_se = TRUE,
  hessian = c("numeric", "analytic"),
  se_method = c("numeric", "opg", "analytic"),
  hess_eps = 1e-05,
  hess_r = 4L
)
```

Arguments

method	Optimizer used by the fitters. Only "BFGS" is implemented and wired through; it is the sole accepted value.
iterlim	Maximum optimizer iterations.
reltol	Relative convergence tolerance.
print_level	Verbosity passed to the optimizer (0 = silent, default = 2 for detailed progress).
draws_hessian	Retained for backward compatibility but currently unused: the random-parameter numeric Hessian is now taken with the same optimization draws that produced the estimate (same-draw curvature), so it no longer resimulates a separate Hessian draw set. Ignored by fit_bnb().

halton_burn	Number of leading Halton points discarded before forming the simulation draws.
n_cores	Worker processes for the optional cluster path (1 = sequential).
compute_se	If FALSE, skip the Hessian and standard errors.
hessian	How fit_bnb() (famoye) computes the Hessian for standard errors: "numeric" (default, numDeriv::hessian()) or "analytic" (the closed-form Famoye (2010) Appendix Hessian). Both freeze the lambda-bounds at the optimum and yield the same observed-information SEs.
se_method	Standard-error method for fit_rpbnb() (the random-parameter model): "numeric" (default) uses the numDeriv::hessian() observed- information Hessian; "analytic" uses the closed-form observed-information Hessian (Famoye (2010) per-draw second derivatives assembled via the Louis mixture formula) – exact and much faster than "numeric" for larger models; "opg" uses the BHHH / outer-product-of-gradients information from the per-observation scores – fastest, but relies on the information-matrix equality so it is unreliable for parameters at a boundary (e.g. a random- coefficient SD estimated near 0). For copula dependence (fit_rpbnb() with dependence = copula(...)), only "opg" (recommended) and "numeric" are available; "analytic" is not implemented for the copula path and errors. Ignored by fit_bnb().
hess_eps, hess_r	Step and Richardson order for numDeriv::hessian() (used only when hessian = "numeric").

Value

An object of class rpbnb_control (a named list).

Examples

```
rpbnb_control(method = "BFGS", iterlim = 200)
rpbnb_control(hessian = "analytic")
```

---

rpbnb_elasticities *Elasticities and semi-elasticities for a random-parameter bivariate NB model*

Description

Continuous elasticities $x_{ij}\,(\partial\mu_i/\partial x_{ij})/\mu_i$ and binary semi-elasticities $\mu_i(x_j=1)/\mu_i(x_j=0)-1$, built on the Monte-Carlo integrated population mean $\mu_i = E_\beta[\exp(x_i'\beta)]$ of an fit_rpbnb() fit. Under fixed coefficients these reduce to $\beta_j E[x_j]$ and $\exp(\beta_j)-1$. Standard errors use a numeric delta method over the equation's mean and log-scale parameters.

Usage

```
rpbnb_elasticities(
  fit,
  which = c("y1", "y2", "both"),
  type = c("AME", "MEM"),
  vars = NULL,
  include_intercept = FALSE,
```

```
  digits = 4,
  print_output = TRUE,
  n_cores = 1L
)
```

Arguments

fit	An rpbnb_fit object from fit_rpbnb().
which	Which margin(s): "y1", "y2", or "both".
type	"AME" (average over the sample) or "MEM" (evaluated at the mean row).
vars	Optional variable names or indices to restrict output.
include_intercept	Logical; include the intercept term.
digits	Number of decimal places for printed output.
print_output	Logical; if FALSE, suppress printing.
n_cores	Number of worker processes for the delta-method standard-error jacobian (1 = sequential, the default). When n_cores > 1, the jacobian's independent per-parameter columns are dispatched across a parallel::makeCluster() cluster (one cluster per call, shared across every equation which computes); results are numerically identical to the sequential path. Falls back to sequential with a warning if the parallel package is unavailable.

Value

A data frame (single margin, invisibly) or a named list of data frames (both), each with columns Name, Estimate, StdErr, z, p, Signif, var_type.

See Also

bnb_elasticities() for fixed-coefficient bnb_fit models.

Examples

```
sim <- simulate_rpbnb(n = 400,
  beta1 = c("(Intercept)" = 0.2, x1 = 0.4),
  beta2 = c("(Intercept)" = 0.1, x1 = -0.3),
  random_1 = list(x1 = list(sd = 0.5)),
  dispersion = c(m1 = 0.4, m2 = 0.5), seed = 1)
fit <- fit_rpbnb(y1 ~ x1, y2 ~ x1, data = sim$data, random_1 = "x1",
                 draws = 100)
rpbnb_elasticities(fit, which = "both", type = "AME")
```

---

rpbnb_marginal_effects

Marginal effects for a random-parameter bivariate NB model

Description

Average marginal effects (AME) or marginal effects at the mean (MEM) for each margin of an `fit_rpbnb()` fit, built on the Monte-Carlo integrated population mean $\mu_i = E_\beta[\exp(x_i'\beta)]$ (the same estimand as `predict.rpbnb_fit()`, reusing the fit's stored draws). Continuous effects are $\partial\mu_i/\partial x_{ij} = \text{mean}_r\, \text{coef}_{rj} \exp(\text{lp}_{ir})$ (the realized coefficient per draw, so random and fixed columns are handled uniformly); binary (0/1) effects are the integrated discrete difference $E[Y|x_j = 1] - E[Y|x_j = 0]$. Standard errors use a numeric delta method over the equation's mean and log-scale parameters.

Usage

```
rpbnb_marginal_effects(
  fit,
  which = c("y1", "y2", "both", "all"),
  type = c("AME", "MEM"),
  vars = NULL,
  include_intercept = FALSE,
  digits = 4,
  print_output = TRUE,
  n_cores = 1L
)
```

Arguments

`fit`	An `rpbnb_fit` object from `fit_rpbnb()`.
`which`	Which margin(s): "y1", "y2", "both", or "all".
`type`	"AME" (average over the sample) or "MEM" (effect at the mean row).
`vars`	Optional variable names or indices to restrict output.
`include_intercept`	Logical; include the intercept term.
`digits`	Number of decimal places for printed output.
`print_output`	Logical; if `FALSE`, suppress printing.
`n_cores`	Number of worker processes for the delta-method standard-error jacobian (1 = sequential, the default). When `n_cores` > 1, the jacobian's independent per-parameter columns are dispatched across a `parallel::makeCluster()` cluster (one cluster per call, shared across every equation `which` computes); results are numerically identical to the sequential path. Falls back to sequential with a warning if the `parallel` package is unavailable.

Value

A data frame (single margin, invisibly) or a named list of data frames (`both`/`all`), each with columns `Name`, `Estimate`, `StdErr`, `z`, `p`, `Signif`, `var_type`.

See Also

`bnb_marginal_effects()` for fixed-coefficient `bnb_fit` models.

Examples

```
sim <- simulate_rpbnb(n = 400,
  beta1 = c("(Intercept)" = 0.2, x1 = 0.4),
  beta2 = c("(Intercept)" = 0.1, x1 = -0.3),
  random_1 = list(x1 = list(sd = 0.5)),
  dispersion = c(m1 = 0.4, m2 = 0.5), seed = 1)
fit <- fit_rpbnb(y1 ~ x1, y2 ~ x1, data = sim$data, random_1 = "x1",
                 draws = 100)
rpbnb_marginal_effects(fit, which = "y1", type = "AME")
```

---

rpbnb_threads *Number of CPU threads available for the multithreaded likelihood*

Description

Number of CPU threads available for the multithreaded likelihood

Usage

```
rpbnb_threads()
```

Value

Integer thread count reported by OpenMP (1 if OpenMP is unavailable).

---

simulate_bnb *Simulate data from the Famoye/Sarmanov bivariate NB distribution*

Description

Generates paired count outcomes `(y1, y2)` from the fixed-parameter Famoye/Sarmanov bivariate NB2 joint PMF:

Usage

```
simulate_bnb(
  n,
  beta1,
  beta2,
  dispersion = c(m1 = 0.5, m2 = 0.5),
  lambda = 0,
  covariates = NULL,
  seed = NULL
)
```

Arguments

n	Number of observations.
beta1, beta2	Named numeric vectors of coefficients for each equation; must include "(Intercept)".
dispersion	Named numeric c(m1 = ..., m2 = ...) NB2 dispersion parameters (variance = mu + m * mu^2).
lambda	Famoye/Sarmanov dependence parameter. Must lie within the valid bounds implied by the marginal means and dispersions.
covariates	Optional data frame of covariates. If NULL, standard- normal columns are generated for every non-intercept coefficient name.
seed	Optional random seed. If NULL (default) the RNG is left untouched and draws continue from the caller's current stream, so repeated calls yield distinct datasets; supply an integer for reproducible output.

Details

$$P(Y_1 = y_1, Y_2 = y_2) = p_1(y_1)\, p_2(y_2)\, [1 + \lambda(e^{-y_1} - c_1)(e^{-y_2} - c_2)]$$

where $c_k = E[e^{-Y_k}]$ under NB2(μ_k, m_k).

Value

A list with:

data data frame with y1, y2, and covariate columns

mu data frame with mu1, mu2 (per-obs conditional means)

true list of true parameters: beta1, beta2, dispersion, lambda

settings list with n and seed

meta list with seed and r_version

Examples

```
sim <- simulate_bnb(n = 500,
  beta1 = c("(Intercept)" = 0.5, x1 = 0.3),
  beta2 = c("(Intercept)" = 0.2, x1 = -0.2),
  dispersion = c(m1 = 0.4, m2 = 0.5), lambda = 0.1, seed = 1)
head(sim$data)
```

---

simulate_rpbnb *Simulate data from a random-parameter bivariate NB process*

Description

Simulate data from a random-parameter bivariate NB process

Usage

```
simulate_rpbnb(
  n,
  beta1,
  beta2,
  random_1 = NULL,
  random_2 = NULL,
  dispersion = c(m1 = 0.5, m2 = 0.5),
  lambda = 0,
  covariates = NULL,
  seed = NULL
)
```

Arguments

n	Number of observations.
beta1, beta2	Named numeric vectors of fixed coefficient means per equation; must include "(Intercept)".
random_1, random_2	Named lists giving random coefficients. Each value is a list with dist (one of "normal", "lognormal", "uniform", "triangular"; default "normal"), scale (or sd) for the dispersion, and sign (-1/1, lognormal only). Means come from beta1/beta2; for a lognormal coefficient the beta entry is the log-location and the realized coefficient is sign * exp(log_location + scale * z).
dispersion	Named numeric c(m1 = ..., m2 = ...) NB2 dispersions.
lambda	Famoye dependence parameter (0 = independent margins).
covariates	Optional data frame of covariates; if NULL, standard-normal columns are generated for every non-intercept name.
seed	Optional random seed. If NULL (default) the RNG is left untouched and draws continue from the caller's current stream, so repeated calls yield distinct datasets; supply an integer for reproducible output.

Value

A list with data (y1, y2, covariates), coef_realized (per-obs coefficients per equation), mu (conditional means), true (parameters), settings, and meta (R/seed/timestamp passed by caller).

Examples

```
sim <- simulate_rpbnb(n = 500,
  beta1 = c("(Intercept)" = 0.2, x1 = 0.4),
  beta2 = c("(Intercept)" = 0.1, x1 = -0.3),
  random_1 = list(x1 = list(sd = 0.5)),
  dispersion = c(m1 = 0.4, m2 = 0.5), seed = 1)
head(sim$data)
```

`simulate_rpbnb_copula` *Simulate data from a copula RP-BNB process*

Description

Simulate data from a copula RP-BNB process

Usage

```
simulate_rpbnb_copula(
  n,
  beta1,
  beta2,
  random_1 = NULL,
  random_2 = NULL,
  dispersion = c(m1 = 0.5, m2 = 0.5),
  copula,
  covariates = NULL,
  seed = NULL
)
```

Arguments

`n`	Number of observations.
`beta1`, `beta2`	Named coefficient means; must include "(Intercept)".
`random_1`, `random_2`	Random-coefficient specs (see `simulate_rpbnb()`).
`dispersion`	Named `c(m1=, m2=)` NB2 dispersions.
`copula`	An `copula()` object giving the family and native parameter `par`.
`covariates`	Optional covariate data frame; NULL -> standard-normal columns.
`seed`	Optional RNG seed.

Value

list(data, mu, true, settings).

Index